SRN : PES2UG24CS455

NAME : SHAM SANTHOSH S

Before changes :

Pylint_before :

```
************* Module supplier_orders
supplier_orders.py:5:0: C0301: Line too long (122/100) (line-too-long)
supplier_orders.py:11:0: C0301: Line too long (131/100) (line-too-long)
supplier_orders.py:1:0: C0114: Missing module docstring (missing-module-docstring)
supplier_orders.py:6:0: C0103: Constant name "order_total" doesn't conform to UPPER_CASE naming style (invalid-name)
supplier_orders.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
supplier_orders.py:9:0: C0103: Function name "placeOrder" doesn't conform to snake_case naming style (invalid-name)
supplier_orders.py:9:15: W0621: Redefining name 'conn' from outer scope (line 4) (redefined-outer-name)
supplier_orders.py:10:4: W0603: Using the global statement (global-statement)
supplier_orders.py:11:17: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
supplier_orders.py:16:0: C0116: Missing function or method docstring (missing-function-docstring)
supplier_orders.py:16:0: C0103: Function name "markReceived" doesn't conform to snake_case naming style (invalid-name)
supplier_orders.py:16:17: W0621: Redefining name 'conn' from outer scope (line 4) (redefined-outer-name)
supplier_orders.py:16:33: W0613: Unused argument 'remarks' (unused-argument)
supplier_orders.py:20:0: C0116: Missing function or method docstring (missing-function-docstring)
supplier_orders.py:20:0: C0103: Function name "notifyWarehouse" doesn't conform to snake_case naming style (invalid-name)
supplier_orders.py:24:0: C0116: Missing function or method docstring (missing-function-docstring)
supplier_orders.py:24:12: W0621: Redefining name 'conn' from outer scope (line 4) (redefined-outer-name)
supplier_orders.py:27:11: W0718: Catching too general exception Exception (broad-exception-caught)
supplier_orders.py:31:0: C0116: Missing function or method docstring (missing-function-docstring)
supplier_orders.py:31:0: C0103: Function name "pendingCount" doesn't conform to snake_case naming style (invalid-name)
supplier_orders.py:31:17: W0621: Redefining name 'conn' from outer scope (line 4) (redefined-outer-name)
supplier_orders.py:35:0: C0116: Missing function or method docstring (missing-function-docstring)

-----------------------------------
Your code has been rated at 2.41/10
```

Bandit_before:

```
python : [main] INFO    profile include tests: None
At line:1 char:1
+ python -m bandit supplier_orders.py *> bandit_before.txt
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: ([main]     INFO    profile include tests: None:String) [], RemoteException
    + FullyQualifiedErrorId : NativeCommandError

[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.15
Run started:2026-10-14 06:29:58.822108+00:00

Test results:
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: .\supplier_orders.py:11:17
10          global order_total
11          conn.execute("INSERT INTO orders (supplier, item, quantity, received) VALUES ('%s', '%s', %d, 0)" % (supplier, item, quantity))
12          order_total+=quantity

--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: .\supplier_orders.py:21:4
20      def notifyWarehouse(supplier):
21          os.system("echo notifying warehouse about " + supplier)
22
```

```
--------------------------------------------------
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
   Location: .\supplier_orders.py:27:4
26              conn.execute("DELETE FROM orders WHERE received = 1")
27          except Exception:
28              pass
29

--------------------------------------------------

Code scanned:
        Total lines of code: 29
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 1
                Medium: 1
                High: 1
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 2
Files skipped (0):
```

Flake8_before:

```
supplier_orders.py:5:80: E501 line too long (122 > 79 characters)
supplier_orders.py:11:80: E501 line too long (131 > 79 characters)
supplier_orders.py:12:16: E225 missing whitespace around operator
supplier_orders.py:32:80: E501 line too long (87 > 79 characters)
```

Supplier_orders.py file before :

import os

import sqlite3

conn = sqlite3.connect(":memory:")

conn.execute("CREATE TABLE orders (id INTEGER PRIMARY KEY, supplier TEXT, item TEXT, quantity INTEGER, received INTEGER)")

order_total = 0

```
def placeOrder(conn, supplier, item, quantity):
  global order_total
  conn.execute("INSERT INTO orders (supplier, item, quantity, received) VALUES ('%s', '%s', %d, 0)" % (supplier, item, quantity))
  order_total+=quantity
  return order_total

def markReceived(conn, order_id, remarks):
  conn.execute("UPDATE orders SET received = 1 WHERE id = ?", (order_id,))

def notifyWarehouse(supplier):
  os.system("echo notifying warehouse about " + supplier)

def cleanup(conn):
  try:
    conn.execute("DELETE FROM orders WHERE received = 1")
  except Exception:
    pass

def pendingCount(conn):
  return conn.execute("SELECT COUNT(*) FROM orders WHERE received = 0").fetchone()[0]

def main():
  placeOrder(conn, "Acme Steel", "rods", 40)
  placeOrder(conn, "Bolt Co", "bolts", 500)
  markReceived(conn, 1, "ok")
  notifyWarehouse("Acme")
  print(pendingCount(conn), order_total)
```

cleanup(conn)

main()

After changes :

notepad pylint_after :

```
************* Module supplier_orders
supplier_orders.py:71:0: C0304: Final newline missing (missing-final-newline)

-----------------------------------------------------------------
Your code has been rated at 9.66/10 (previous run: 2.41/10, +7.24)
```

notepad bandit_after:

```
python : [main] INFO    profile include tests: None
At line:1 char:1
+ python -m bandit supplier_orders.py *> bandit_after.txt
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : NotSpecified: ([main]     INFO    profile include tests: None:String) [], RemoteException
    + FullyQualifiedErrorId : NativeCommandError

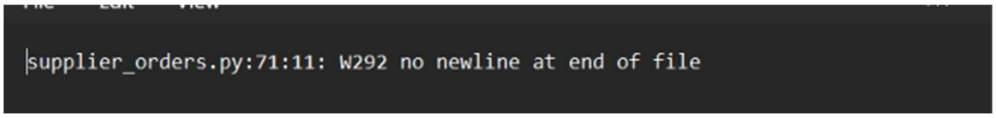
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.15
Run started:2026-10-14 06:47:05.052680+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 54
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

notepad flake8_after.txt



Suppelier_orders code after changes :

```
"""Track supplier purchase orders and warehouse deliveries."""

import sqlite3

def create_database():
    """Create an in-memory database and its orders table."""
    connection = sqlite3.connect(":memory:")
    connection.execute(
        """CREATE TABLE orders (
            id INTEGER PRIMARY KEY,
            supplier TEXT,
            item TEXT,
            quantity INTEGER,
            received INTEGER
        )"""
    )
    return connection

def place_order(connection, supplier, item, quantity):
    """Insert an order and return its quantity."""
    connection.execute(
```

```
      """INSERT INTO orders (supplier, item, quantity, received)
        VALUES (?, ?, ?, 0)""",
      (supplier, item, quantity),
    )
    return quantity

def mark_received(connection, order_id):
    """Mark the specified order as received."""
    connection.execute(
      "UPDATE orders SET received = 1 WHERE id = ?",
      (order_id,),
    )

def notify_warehouse(supplier):
    """Print a warehouse notification for the supplier."""
    print(f"notifying warehouse about {supplier}")

def cleanup(connection):
    """Remove orders that have already been received."""
    connection.execute("DELETE FROM orders WHERE received = 1")

def pending_count(connection):
    """Return the number of orders that are still pending."""
    result = connection.execute(
      "SELECT COUNT(*) FROM orders WHERE received = 0"
    ).fetchone()
    return result[0]
```

```
def main():
    """Run a short demonstration of the purchase-order tracker."""
    connection = create_database()
    order_total = 0
    order_total += place_order(connection, "Acme Steel", "rods", 40)
    order_total += place_order(connection, "Bolt Co", "bolts", 500)
    mark_received(connection, 1)
    notify_warehouse("Acme")
    print(pending_count(connection), order_total)
    cleanup(connection)
    connection.close()

if __name__ == "__main__":
    main()
```